- ***4*** The actions are stored as integers in the replay memory, so we convert to a one-hot encoded vector.

None of these components have complicated architectures. They're fairly mundane, but together they form a powerful system. Now we need to include our DQN model, which is a simple set of a few convolutional layers.

Listing 8.7. Deep Q-network

```
class Qnetwork(nn.Module):
    def __init__(self):
        super(Qnetwork, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=32,
    kernel_size=(3,3), stride=2, padding=1)
        self.conv2 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=2,
padding=1)
        self.conv3 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=2,
padding=1)
        self.conv4 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=2,
padding=1)
        self.linear1 = nn.Linear(288,100)
        self.linear2 = nn.Linear(100,12)

    def forward(self,x):
        x = F.normalize(x)
        y = F.elu(self.conv1(x))
        y = F.elu(self.conv2(y))
        y = F.elu(self.conv3(y))
        y = F.elu(self.conv4(y))
        y = y.flatten(start_dim=2)
        y = y.view(y.shape[0], -1, 32)
        y = y.flatten(start_dim=1)
        y = F.elu(self.linear1(y))
        y = self.linear2(y)            1
        return y
```

- ***1*** The output is of shape N x 12.

We've covered the ICM components; now let's put them together. We're going to define a function that accepts (S_t,a_t,S_{t+1}) and returns the forward-model prediction error and the inverse-model error. The forward-model error will be used not only to backpropagate and train the forward model but also as the intrinsic reward for the DQN. The inverse-model error is